

使用 ML Kit (Android) 偵測圖片中的物件，建構視覺化產品搜尋功能

來源：<https://codelabs.developers.google.com/product-search-odt-android?hl=zh-tw>

步驟數：7

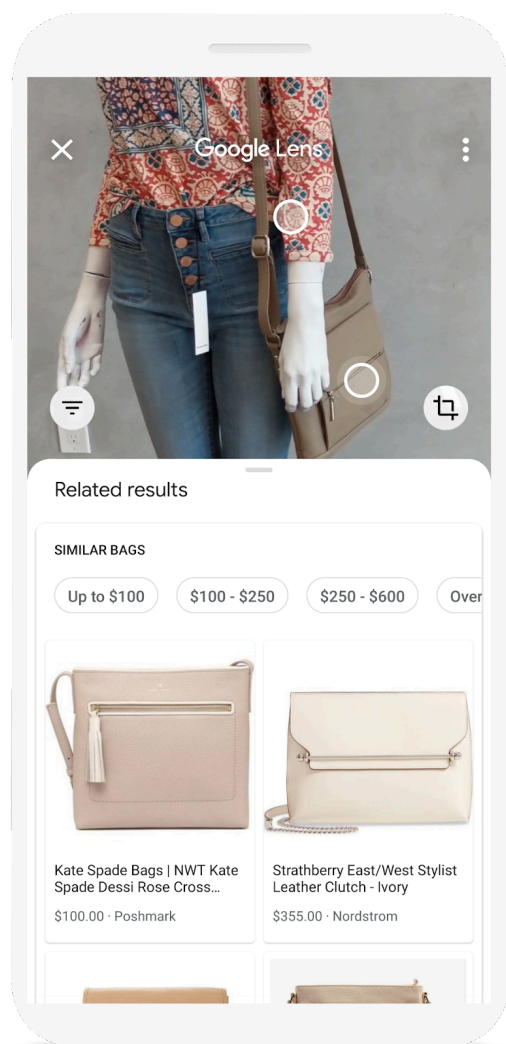
在本程式碼研究室中，您將使用 ML Kit 建構 Android 應用程式，這個應用程式會使用裝置端機器學習技術偵測圖片中的物件，然後讓使用者進行視覺化的產品搜尋。

目錄

1. 事前準備
2. 做好準備
3. 將 ML Kit 物件偵測與追蹤 API 新增至專案
4. 執行範例應用程式
5. 新增裝置端物件偵測功能
6. 後續處理偵測結果
7. 恭喜！

步驟 1 · 約 0 分鐘

1. 事前準備



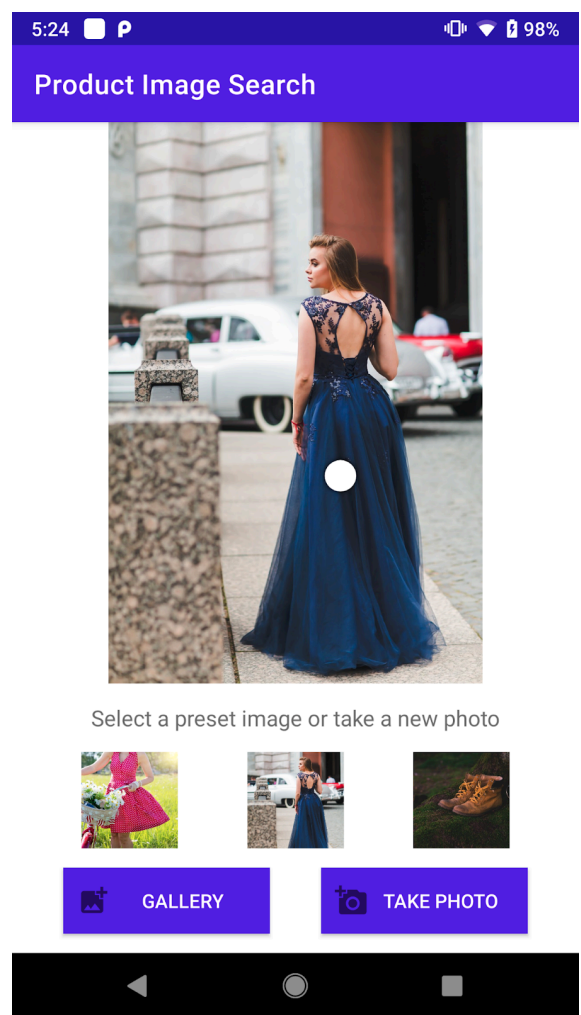
你是否看過 Google 智慧鏡頭的示範影片，只要將手機鏡頭對準物體，就能找到線上購買管道？如要瞭解如何在應用程式中加入相同功能，請參閱本程式碼研究室。這是學習路徑的一部分，可教導您如何在行動應用程式中建構產品圖片搜尋功能。

在本程式碼研究室中，您將瞭解建構產品圖片搜尋功能的第一步：如何偵測圖片中的物件，並讓使用者選擇要搜尋的物件。您將使用 **ML Kit 物件偵測和追蹤** 功能建構這項功能。

如要瞭解後續步驟，包括如何使用 **Vision API Product Search** 建構產品搜尋後端，請參閱[學習路徑](#)。

建構項目

- 在本程式碼研究室中，您將使用 ML Kit 建構 Android 應用程式。您的應用程式會使用 ML Kit 物件偵測和追蹤 API 偵測指定圖片中的物件。接著，使用者會在我們的產品資料庫中選取要搜尋的物件。
- 最後，您應該會看到類似右側圖片的畫面。



課程內容

- 如何將 ML Kit SDK 整合至 Android 應用程式
- ML Kit 物件偵測與追蹤 API

軟硬體需求

- 最新版 Android Studio (4.1.2 以上版本)
- Android Studio 模擬器或實體 Android 裝置
- [程式碼範例](#)
- 具備 Kotlin Android 開發作業的基本知識

本程式碼研究室著重於 ML Kit。我們不會研究其他概念和程式碼區塊，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

2. 做好準備

下載程式碼

點選下方連結即可下載這個程式碼研究室的所有程式碼：

將下載的 ZIP 檔案解壓縮。這會解壓縮根資料夾 (`odml-pathways-main`)，其中包含您需要的所有資源。在本程式碼研究室中，您只需要 `product-search/codelab1/android` 子目錄中的來源。

mlkit-android 存放區中的物件偵測子目錄包含兩個目錄：

- 
- **starter**：本程式碼研究室的範例程式碼，您將以這個程式碼為基礎進行建構。
- 
- **final**：完成的範例應用程式程式碼。
-

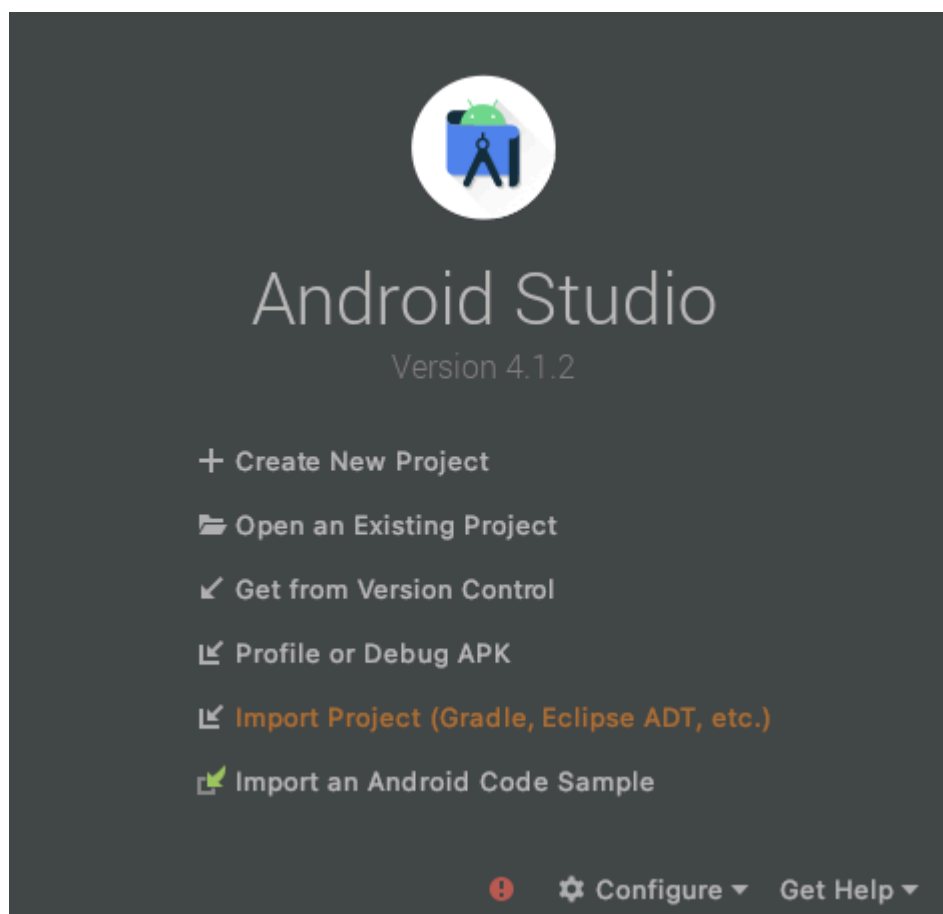
步驟 3 · 約 4 分鐘

3. 將 ML Kit 物件偵測與追蹤 API 新增至專案

將應用程式匯入 Android Studio

首先，請將範例應用程式匯入 Android Studio。

前往 Android Studio，選取「Import Project (Gradle, Eclipse ADT, etc.)」，然後選擇先前下載的原始碼中的 **starter** 資料夾。



新增 ML Kit 物件偵測與追蹤的依附元件

您可以使用 ML Kit 依附元件，在應用程式中整合 ML Kit ODT SDK。

前往專案的 `app/build.gradle` 檔案，確認依附元件是否已存在：

build.gradle

```
dependencies {  
    // ...  
    implementation 'com.google.mlkit:object-detection:16.2.4'  
}
```

將專案與 Gradle 檔案同步

為確保應用程式可使用所有依附元件，您應在此時將專案與 Gradle 檔案同步處理。

從 Android Studio 工具列選取「Sync Project with Gradle Files」（



)。

(如果這個按鈕已停用，請確認您只匯入 **starter/app/build.gradle**，而非整個存放區)。

步驟 4 · 約 3 分鐘

4. 執行範例應用程式

您已將專案匯入 Android Studio，並新增 ML Kit 物件偵測和追蹤功能的依附元件，現在可以首次執行應用程式。

透過 USB 將 Android 裝置連線至主機，或[啟動 Android Studio 模擬器](#)，然後按一下 Android Studio 工具列中的「Run」(

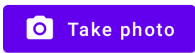


)。

執行並探索應用程式

Android 裝置上應會啟動應用程式。其中包含一些樣板程式碼，可讓您擷取相片或選取預設圖片，並將其提供給您將在本程式碼研究室中建構的物件偵測和追蹤管道。編寫程式碼前，請先稍微探索一下應用程式：

首先，底部有一個「按鈕」(



)，

- 啟動裝置/模擬器整合的相機應用程式
- 在**相機應用程式**中拍照
- 在**範例應用程式**中接收擷取的圖片
- 顯示圖片

試試「拍照」按鈕。按照提示拍攝相片，然後**接受**相片，並觀察相片顯示在**範例應用程式**中。

其次，你可以從 3 張預設圖片中選擇。如果您在 Android 模擬器上執行，稍後可以使用這些圖片測試物件偵測程式碼。

1. 從 3 張預設圖片中**選取**一張。
2. 確認圖片是否顯示在較大的檢視畫面中。

5:27



98%

Product Image Search



Select a preset image or take a new photo



GALLERY



TAKE PHOTO



5. 新增裝置端物件偵測功能

在這個步驟中，您將為範例應用程式新增功能，偵測圖片中的物件。如上一步所示，範例應用程式包含樣板程式碼，可透過裝置上的相機應用程式拍照。如果您在 Android 模擬器上執行程式碼研究室，也可以試用應用程式中的 3 張預設圖片進行物件偵測。

選取圖片時 (無論是預設圖片或使用相機應用程式拍攝的相片)，樣板程式碼都會將圖片解碼為 `Bitmap` 執行個體、顯示在畫面上，並使用圖片呼叫 `runObjectDetection` 方法。

在這個步驟中，您將在 `runObjectDetection` 方法中加入程式碼，執行物件偵測！

設定及執行圖片的裝置端物件偵測

設定 ML Kit ODT 只需要 3 個簡單步驟，以及 3 個 API

- 準備圖片：`InputImage`
- 建立偵測器物件：`ObjectDetection.getClient(options)`
- 連結上述 2 個物件：`process(image)`

您可以在 `MainActivity.kt` 檔案的 `**runObjectDetection(bitmap: Bitmap)**` 函式中完成這些作業。

```
/**
 * ML Kit Object Detection Function
 */
private fun runObjectDetection(bitmap: Bitmap) {
}
```

目前函式是空的。請繼續進行下列步驟，整合 *ML Kit ODT*！過程中，Android Studio 會提示您新增必要的匯入項目

- `com.google.mlkit.vision.common.InputImage`
- `com.google.mlkit.vision.objects.ObjectDetection`
- `com.google.mlkit.vision.objects.defaults.ObjectDetectorOptions`

步驟 1：建立 `InputImage`

ML Kit 提供簡單的 API，可從 `Bitmap` 建立 `InputImage`。接著，您就能將 `InputImage` 提供給 ML Kit API。

```
// Step 1: create ML Kit's InputImage object
val image = InputImage.fromBitmap(bitmap, 0)
```

新增上述程式碼至 `runObjectDetection(bitmap:Bitmap)` 頂端。

步驟 2：建立偵測工具執行個體

ML Kit 採用 *建構工具設計模式*，您會將設定傳遞至建構工具，然後從中取得 *偵測工具*。有 3 個設定選項 (程式碼研究室使用粗體選項)：

- 偵測器模式 (**單一圖片**或串流)
- 偵測模式 (**單一**或 **多個** 物件偵測)
- 分類模式 (**開啟**或關閉)

本程式碼研究室適用於單一圖片 - 多個物件偵測和分類，讓我們開始吧：

```
// Step 2: acquire detector object
val options = ObjectDetectorOptions.Builder()
    .setDetectorMode(ObjectDetectorOptions.SINGLE_IMAGE_MODE)
    .enableMultipleObjects()
    .enableClassification()
    .build()
val objectDetector = ObjectDetection.getClient(options)
```

步驟 3：將圖片提供給偵測器

物件偵測和分類是異步處理：

- 將圖片傳送至偵測器 (透過 `process()`)
- 偵測器會盡力偵測
- 偵測器會透過回呼將結果回報給您

下列程式碼會執行這項操作 (複製並附加至 `fun runObjectDetection(bitmap: Bitmap)`)： 內現有的程式碼)：

```
// Step 3: feed given image to detector and setup callback
objectDetector.process(image)
    .addOnSuccessListener {
        // Task completed successfully
        debugPrint(it)
    }
    .addOnFailureListener {
        // Task failed with an exception
        Log.e(TAG, it.message.toString())
    }
```

完成後，偵測器會透過下列方式通知你：

1. 偵測到的物件總數
2. 系統會使用
 - `trackingId`：用於跨影格追蹤的整數 (本程式碼研究室不會使用)
 - `boundingBox`：物件的定界框
 - `labels`：偵測到的物件標籤清單 (僅在啟用分類功能時顯示)
 - `index` (取得這個標籤的索引)
 - `text` (取得這個標籤的文字，包括「時尚商品」、「食品」、「居家用品」、「地點」、「植物」)
 - `confidence` (介於 0.0 到 1.0 的浮點值，1.0 代表 100%)

您可能已注意到，程式碼會使用 `debugPrint()` 將偵測到的結果輸出至 Logcat。將其新增至 `MainActivity` 類別：

```
private fun debugPrint(detectedObjects: List<DetectedObject>) {
    detectedObjects.forEachIndexed { index, detectedObject ->
        val box = detectedObject.boundingBox

        Log.d(TAG, "Detected object: $index")
        Log.d(TAG, " trackingId: ${detectedObject.trackingId}")
        Log.d(TAG, " boundingBox: (${box.left}, ${box.top}) - (${box.right},${box.bottom})")
        detectedObject.labels.forEach {
            Log.d(TAG, " categories: ${it.text}")
            Log.d(TAG, " confidence: ${it.confidence}")
        }
    }
}
```

現在您可以接受圖片進行偵測了！

在 Android Studio 工具列中按一下「Run」 (



)，執行程式碼研究室。請嘗試選取預設圖片或拍照，然後查看 IDE 內的 *logcat* 視窗(

☰ 6: Logcat

)。畫面應如下所示：

```
D/MLKit Object Detection: Detected object: 0
D/MLKit Object Detection: trackingId: null
D/MLKit Object Detection: boundingBox: (481, 2021) - (2426,3376)
D/MLKit Object Detection: categories: Fashion good
D/MLKit Object Detection: confidence: 0.90234375
D/MLKit Object Detection: Detected object: 1
D/MLKit Object Detection: trackingId: null
D/MLKit Object Detection: boundingBox: (2639, 2633) - (3058,3577)
D/MLKit Object Detection: Detected object: 2
D/MLKit Object Detection: trackingId: null
D/MLKit Object Detection: boundingBox: (3, 1816) - (615,2597)
D/MLKit Object Detection: categories: Home good
D/MLKit Object Detection: confidence: 0.75390625
```

也就是說，偵測器偵測到 3 個物件：

- 類別為「時尚商品」和「居家用品」。
- 由於第 2 個是未知類別，因此不會傳回任何類別。
- 無 `trackingId` (因為這是單一圖片偵測模式)
- `boundingBox` 矩形內的位置 (例如 (481, 2021) - (2426, 3376))
- 偵測工具相當確定第 1 項是**時尚商品** (90%) (是洋裝)

從技術上來說，您目前已取得所有必要項目，可讓 ML Kit 物件偵測功能正常運作！恭喜！

沒錯，在 UI 方面，您仍處於開始階段，但可以利用 UI 上偵測到的結果 (例如繪製出定界框)，打造更優質的體驗。下一步是將偵測到的結果視覺化！

6. 後續處理偵測結果

在先前的步驟中，您已將偵測結果列印到 *logcat* 中，簡單又快速。

在本節中，您將使用圖片中的結果：

- 在圖片上繪製定界框
- 在定界框內繪製類別名稱和信賴度

瞭解視覺化公用程式

程式碼研究室中提供一些樣板程式碼，協助您以視覺化方式呈現偵測結果。運用這些公用程式，簡化視覺化程式碼：

- `class ImageClickableView` 這是圖片檢視區塊類別，提供一些實用的工具，可顯示偵測結果並與之互動。
- `fun drawDetectionResults(results: List<DetectedObject>)` 這個方法會在偵測到的每個物件中心繪製白色圓圈。
- `fun setOnObjectClickListener(listener: ((objectImage: Bitmap) -> Unit))`：這是回呼，用於接收僅包含使用者輕觸物件的裁剪圖片。您會在後續的程式碼研究室中，將這張裁剪後的圖片傳送至圖片搜尋後端，取得視覺上相似的結果。在本程式碼研究室中，您還不會使用這個方法。

顯示 ML Kit 偵測結果

使用視覺化公用程式，在輸入圖片上顯示 ML Kit 物件偵測結果。

前往呼叫 `debugPrint()` 的位置，並在下方新增下列程式碼片段：

```
runOnUiThread {  
    viewBinding.ivPreview.drawDetectionResults(it)  
}
```

開始執行

現在請按一下 Android Studio 工具列中的「Run」 (



)。

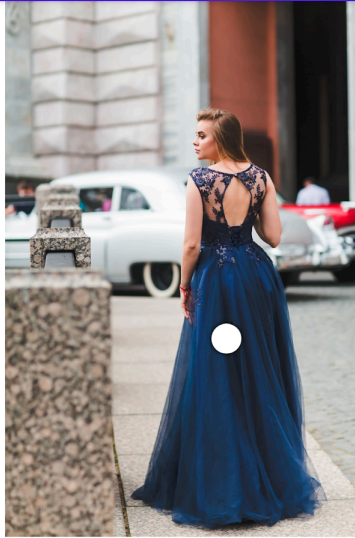
應用程式載入後，按一下相機圖示按鈕，將相機對準物體，然後拍照、接受相片 (在「相機」應用程式中)，或輕觸任何預設圖片。您應該會看到偵測結果；再次按下按鈕或選取其他圖片，重複幾次，即可體驗最新的 ML Kit ODT！

5:24



98%

Product Image Search



Select a preset image or take a new photo



GALLERY



TAKE PHOTO



步驟 7 · 約 0 分鐘

7. 恭喜！

您已使用 ML Kit 將物件偵測功能新增至應用程式：

- **3 個步驟和 3 個 API**
- 建立輸入圖片
- 建立偵測工具
- 將圖片傳送給偵測工具

這樣就能開始使用！

涵蓋內容

- 如何在 Android 應用程式中加入 ML Kit 物件偵測與追蹤功能
- 如何使用 ML Kit 的裝置端物件偵測和追蹤功能，偵測圖片中的物件

後續步驟

- 請參閱這項[程式碼研究室](#)，瞭解如何將偵測到的物件傳送至 Product Search 後端，並顯示搜尋結果
- 使用 ML Kit ODT 探索更多內容，透過更多圖片和即時影像體驗偵測和分類準確率及效能
- 請參閱「[進一步瞭解物件偵測](#)」學習路徑，瞭解如何訓練自訂模型
- 請參閱有關物件偵測 [即時攝影機](#)和[靜態圖片](#)的 Material Design 建議
- 在自己的 Android 應用程式中套用 ML Kit ODT

瞭解詳情

- <https://developers.google.com/ml-kit/vision/object-detection>
-